

資料結構專題報告-餐飲管理系統

4111029004 許家睿

專案簡介

專案名稱：餐飲管理系統

主要功能：

- 管理者可管理菜單與查看訂單
- 顧客可瀏覽菜單、點餐並查看訂單收據

使用方法：

執行 `main.py`

系統架構

- 使用者模式
 - 管理者模式
 - 顧客模式
- 使用的資料結構方法
 - Linked List
 - AVLTree
 - Stack
 - 遞迴

admin.py (管理者帳號系統)

```
class AdminNode:
    def __init__(self, username, password):
        self.username = username
        self.password = password
        self.next = None
```

當新增一個管理者時，會建立一個新的 AdminNode

username：管理者帳號

password：管理者密碼

```
class AdminLinkedList:
    def __init__(self):
        self.head = None

    def register(self, username, password):
        if self.search(username):
            print("帳號已存在，請使用其他帳號")
            return False

        new_admin = AdminNode(username, password)
        new_admin.next = self.head
        self.head = new_admin
        print("管理者註冊成功")
        return True
```

管理所有管理者的節點

def register：註冊新管理者帳號

search 方法確認帳號是否重複

成功註冊回傳 True，失敗回傳 False

```
def search(self, username):
    current = self.head
    while current:
        if current.username == username:
            return current
        current = current.next
    return None
```

def search：
搜尋帳號是否存在

```
--- 管理者模式 ---
1. 管理者登入
2. 管理者註冊
3. 返回主選單
請選擇功能：2
請輸入新管理者帳號：admin
請輸入新管理者密碼：1
帳號已存在，請使用其他帳號
```

admin.py

```
def login(self, username, password):
    current = self.head
    while current:
        if current.username == username and current.password == password:
            print("管理者登入成功")
            return True
        current = current.next
    print("帳號或密碼錯誤")
    return False
```

def login : 驗證管理者帳號密碼

登入成功: 顯示"管理者登入成功", 回傳 True
登入失敗: 顯示"帳號或密碼錯誤", 回傳 False

```
def display_admins(self):
    current = self.head
    print("\n--- 管理者列表 ---")
    while current:
        print(f" - {current.username}")
        current = current.next
```

def display_admins : 顯示所有已註冊的管理者帳號

```
--- 管理者面板 ---
1. 檢視所有訂單
2. 顯示管理者帳號
3. AVLTree菜單管理
4. 登出
請選擇功能: 2

--- 管理者列表 ---
- 1
- admin
```

avltree.py (菜單項目管理)

```
class AVLNode:
    def __init__(self, key, price, is_drink):
        self.key = key
        self.price = price
        self.is_drink = is_drink
        self.height = 1
        self.left = None
        self.right = None
```

key：菜單項目名稱
price：菜單項目的價格
is_drink：標記是否為飲料

```
class AVLTree:
    def __init__(self):
        self.root = None

    def get_height(self, node):
        return node.height if node else 0

    def get_balance(self, node):
        return self.get_height(node.left) - self.get_height(node.right) if node else 0
```

def get_height :
獲取節點高度

def get_balance :
計算節點平衡因子

avltree.py

```
def rotate_right(self, y):
    x = y.left
    T2 = x.right
    x.right = y
    y.left = T2
    y.height = max(self.get_height(y.left), self.get_height(y.right)) + 1
    x.height = max(self.get_height(x.left), self.get_height(x.right)) + 1
    return x

def rotate_left(self, x):
    y = x.right
    T2 = y.left
    y.left = x
    x.right = T2
    x.height = max(self.get_height(x.left), self.get_height(x.right)) + 1
    y.height = max(self.get_height(y.left), self.get_height(y.right)) + 1
    return y
```

def rotate_right :
執行右旋轉維持平衡

def rotate_left :
執行左旋轉維持平衡

avltree.py

def insert: 新增菜單項目

遞迴找到插入的位置

根據不同情況執行左右旋轉

```
def add_item(self, key, price, is_drink=False):  
    self.root = self.insert(self.root, key, price, is_drink)
```

def add_item:
將新菜單項目新增到 AVL 樹中

更新self.root

```
def insert(self, node, key, price, is_drink):  
    if not node:  
        return AVLNode(key, price, is_drink)  
  
    if key < node.key:  
        node.left = self.insert(node.left, key, price, is_drink)  
    elif key > node.key:  
        node.right = self.insert(node.right, key, price, is_drink)  
    else:  
        return node  
  
    node.height = 1 + max(self.get_height(node.left), self.get_height(node.right))  
  
    balance = self.get_balance(node)  
  
    if balance > 1 and key < node.left.key:  
        return self.rotate_right(node)  
  
    if balance < -1 and key > node.right.key:  
        return self.rotate_left(node)  
  
    if balance > 1 and key > node.left.key:  
        node.left = self.rotate_left(node.left)  
        return self.rotate_right(node)  
  
    if balance < -1 and key < node.right.key:  
        node.right = self.rotate_right(node.right)  
        return self.rotate_left(node)  
    return node
```


avltree.py

def remove :

刪除指定節點

遞迴搜尋節點

分別處理葉子節點、單子節點、雙子節點

```
def remove_item(self, key):  
    self.root = self.remove(self.root, key)
```

def remove_item :

移除特定的菜單項目

更新 self.root

```
def remove(self, node, key):  
    if not node:  
        return node  
  
    if key < node.key:  
        node.left = self.remove(node.left, key)  
    elif key > node.key:  
        node.right = self.remove(node.right, key)  
    else:  
        if not node.left:  
            return node.right  
        elif not node.right:  
            return node.left  
  
        temp = self.get_min_value_node(node.right)  
        node.key = temp.key  
        node.price = temp.price  
        node.is_drink = temp.is_drink  
        node.right = self.remove(node.right, temp.key)  
  
    if not node:  
        return node  
  
    node.height = 1 + max(self.get_height(node.left), self.get_height(node.right))  
    balance = self.get_balance(node)  
  
    if balance > 1 and self.get_balance(node.left) >= 0:  
        return self.rotate_right(node)  
    if balance > 1 and self.get_balance(node.left) < 0:  
        node.left = self.rotate_left(node.left)  
        return self.rotate_right(node)  
    if balance < -1 and self.get_balance(node.right) <= 0:  
        return self.rotate_left(node)  
    if balance < -1 and self.get_balance(node.right) > 0:  
        node.right = self.rotate_right(node.right)  
        return self.rotate_left(node)  
  
    return node
```

avltree.py

```
def get_min_value_node(self, node):  
    current = node  
    while current.left:  
        current = current.left  
    return current
```

def get_min_value_node : 尋找子樹中鍵值最小的節點

```
def search(self, node, key):  
    if not node or node.key == key:  
        return node  
  
    if key < node.key:  
        return self.search(node.left, key)  
    else:  
        return self.search(node.right, key)
```

def search : 搜尋特定鍵值的節點

```
def inorder(self, node):  
    if node:  
        self.inorder(node.left)  
        print(f"{node.key}: ${node.price}")  
        self.inorder(node.right)
```

```
def display_menu(self):  
    print("\n----- 菜單 -----")  
    self.inorder(self.root)
```

def inorder, def display_menu :
中序遍歷AVLTree，由小到大顯示所有節點

menu.py (菜單管理-調用 avltree.py)

```
from avltree import AVLTree

class Menu:
    def __init__(self):
        # self.menu_items = {}
        self.menu_tree = AVLTree()
```

class Menu : 負責管理菜單項目
所有菜單項目的新增、刪除、搜尋和顯示都透過AVLTree操作

```
def add_item(self, item_name, price, is_drink=False):
    # self.menu_items[item_name] = {'price': price, 'is_drink': is_drink}
    self.menu_tree.add_item(item_name, price, is_drink)
```

def add_item : 將新的菜單項目加入到AVLTree中

```
def remove_item(self, item_name):
    return self.menu_tree.remove_item(item_name)
```

def remove_item : 從AVLTree中移除指定菜單項目

```
def find_item(self, item_name):
    return self.menu_tree.search(self.menu_tree.root, item_name)
```

def find_item : 在菜單中搜尋指定項目

```
def display_menu(self):
    self.menu_tree.display_menu()
```

def display_menu : 顯示所有菜單項目，中序遍歷AVLTree

order.py (顧客訂單管理)

```
class Node:
    def __init__(self, item_name, quantity, price, ice_level=None):
        self.item_name = item_name
        self.quantity = quantity
        self.price = price
        self.ice_level = ice_level
        self.next = None
```

item_name : 項目名稱
quantity : 數量
price : 單價
ice_level : 冰量選擇

```
class Order:
    all_orders = []

    def __init__(self, menu):
        self.head = None
        self.menu = menu
```

class Order : 管理顧客的訂單
all_orders :
儲存所有完成的訂單記錄
每張訂單是一個單向鏈結串列

order.py

```
def add_item(self, item_name, quantity):
    node = self.menu.find_item(item_name)
    if not node:
        print("菜單中沒有這個項目")
        return

    price = node.price
    is_drink = node.is_drink

    if is_drink:
        ice_level = self.get_ice_level(item_name)
    else:
        ice_level = None

    current = self.head
    while current:
        if current.item_name == item_name and current.ice_level == ice_level:
            current.quantity += quantity
            return
        current = current.next

    new_node = Node(item_name, quantity, price, ice_level)
    new_node.next = self.head
    self.head = new_node
```

def add_item : 將新項目新增到訂單中

透過menu.find_item尋找項目是否存在於菜單

如果是飲料，呼叫get_ice_level()取得冰量

檢查是否已存在相同項目

如果存在：增加數量

如果不存在：創建一個新節點並插入鏈結串列的頭部

order.py

```
def get_ice_level(self, item_name):
    print("請選擇冰量:")
    print("1. 正常")
    print("2. 少冰")
    print("3. 去冰")
    choice = input("請輸入選擇的數字: ")
    ice_levels = {1: ("正常", ""), 2: ("少冰", "*"), 3: ("去冰", "^")}

    try:
        selected_ice_level = ice_levels[int(choice)]
        return selected_ice_level
    except KeyError:
        print("選擇無效，使用預設值 '正常'")
        return ("正常", "")
```

def get_ice_level: 讓使用者選擇飲料的冰量

1. 顯示冰量選單
2. 使用者輸入選擇
3. 輸入有效: 回傳選擇的冰量
4. 輸入無效: 回傳預設值 ("正常", "")

```
def remove_last_item(self):
    if self.head:
        removed_item = self.head
        self.head = self.head.next
        print(f"已移除最後一項: {removed_item.item_name} x {removed_item.quantity}")
    else:
        print("訂單為空，無法移除項目。")
```

def remove_last_item: 移除最新加入的項目 (head 節點)

order.py

```
def calculate_total(self):
    total = 0
    current = self.head
    while current:
        total += current.quantity * current.price
        current = current.next
    return total
```

def calculate_total: 計算訂單的總金額

```
def display_order(self):
    print("----- 收據 -----")
    current = self.head
    if not current:
        print("訂單為空")
        return

    while current:
        ice_level_str = current.ice_level[1] if current.ice_level else ""
        print(f"{current.item_name}{ice_level_str}: {current.quantity} x ${current.price:.0f}")
        current = current.next

    print(f"總金額: ${self.calculate_total():.0f}")
```

def display_order:
顯示收據和總金額

order.py

```
def save_receipt(self):
    if not self.head:
        print("沒有訂單可儲存")
        return

    order_summary = {
        "id": len(Order.all_orders) + 1,
        "items": [],
        "total": self.calculate_total()
    }

    current = self.head
    while current:
        order_summary["items"].append({
            "item_name": current.item_name,
            "quantity": current.quantity,
            "price": current.price,
            "ice_level": current.ice_level
        })
        current = current.next

    Order.all_orders.append(order_summary)
    print(f"訂單已儲存 (訂單編號: {order_summary['id']})")
    self.head = None
```

def save_receipt : 將當前訂單的收據儲存到訂單紀錄中

id : 訂單編號

items : 空列表，用於存放每個訂單項目的詳細資訊

total : 呼叫self.calculate_total()計算訂單的總金額

遍歷鏈結串列，加入每個項目到 items

儲存到 Order.all_orders後清空
當前訂單

```
----- 收據 -----
Tea: 1 x $25
Burger: 1 x $150
總金額: $175
訂單已儲存 (訂單編號: 1)
感謝您的點餐
```


main.py (提供管理者模式和顧客模式功能)

```
import sys
from menu import Menu
from order import Order
from admin import AdminLinkedList
from avltree import AVLTree

admin_list = AdminLinkedList()
admin_list.register("admin", "admin")
```

匯入需用到的模組

AdminLinkedList: 建立管理者帳號鏈結串列

register: 預設註冊一個帳號密碼為admin的管理者帳號

```
def admin_avl_menu(menu):
    while True:
        print("\n--- AVLTree菜單管理 ---")
        print("1. 新增菜單項目")
        print("2. 刪除菜單項目")
        print("3. 搜尋菜單項目")
        print("4. 顯示菜單項目")
        print("5. 返回管理者面板")

        choice = input("請選擇功能: ")
```

顯示菜單管理的選項，讓管理者輸入數字選擇不同的功能

```
--- 管理者面板 ---
1. 檢視所有訂單
2. 顯示管理者帳號
3. AVLTree菜單管理
4. 登出
請選擇功能: 3
```

```
--- AVLTree菜單管理 ---
1. 新增菜單項目
2. 刪除菜單項目
3. 搜尋菜單項目
4. 顯示菜單項目
5. 返回管理者面板
請選擇功能: |
```

main.py

輸入1: 新增菜單項目

輸入名稱、價格、輸入 y 或 n 來標記是否為飲料

輸入2: 刪除菜單項目

輸入3: 尋找菜單項目

輸入4: 顯示菜單項目

輸入5: 返回管理員面板

```
--- AVLTree菜單管理 ---
1. 新增菜單項目
2. 刪除菜單項目
3. 搜尋菜單項目
4. 顯示菜單項目
5. 返回管理者面板
請選擇功能: 1
請輸入菜單項目名稱: Water
請輸入價格: 1
是否為飲料?(y/n): y
已將 Water 新增至菜單，價格為 $1
```

```
choice = input("請選擇功能: ")
if choice == '1':
    item_name = input("請輸入菜單項目名稱: ")
    try:
        price = int(input("請輸入價格: "))
        is_drink = input("是否為飲料?(y/n): ").lower() == 'y'
        menu.add_item(item_name, price, is_drink)
        print(f"已將 {item_name} 新增至菜單，價格為 ${price:.0f}")
    except ValueError:
        print("輸入無效，請重新輸入")
elif choice == '2':
    item_name = input("請輸入要刪除的菜單項目名稱: ")
    node = menu.find_item(item_name)
    if node:
        menu.remove_item(item_name)
        print(f"已刪除項目: {item_name}")
    else:
        print("菜單中沒有該項目")
elif choice == '3':
    item_name = input("請輸入要搜尋的菜單項目名稱: ")
    node = menu.find_item(item_name)
    if node:
        print(f"{node.key}: ${node.price}")
    else:
        print("菜單中沒有該項目")
elif choice == '4':
    menu.display_menu()
elif choice == '5':
    break
else:
    print("選項無效，請重新選擇")
```

main.py

def admin_mode: 管理者模式的主介面

輸入1: 管理者登入

輸入2: 管理者註冊

若註冊成功則直接跳到登入畫面

輸入3: 返回主選單

```
--- 管理者模式 ---
1. 管理者登入
2. 管理者註冊
3. 返回主選單
請選擇功能: 1
請輸入管理者帳號: admin
請輸入管理者密碼: admin
管理者登入成功
```

```
--- 管理者面板 ---
1. 檢視所有訂單
2. 顯示管理者帳號
3. AVLTree菜單管理
4. 登出
請選擇功能: |
```

```
def admin_mode(menu, order):
    while True:
        print("\n--- 管理者模式 ---")
        print("1. 管理者登入")
        print("2. 管理者註冊")
        print("3. 返回主選單")
        admin_choice = input("請選擇功能: ")

        if admin_choice == '1':
            username = input("請輸入管理者帳號: ")
            password = input("請輸入管理者密碼: ")
            if admin_list.login(username, password):
                admin_panel(menu, order)
        elif admin_choice == '2':
            username = input("請輸入新管理者帳號: ")
            password = input("請輸入新管理者密碼: ")
            if admin_list.register(username, password):
                print("\n--- 管理者登入 ---")
                username = input("請輸入管理者帳號: ")
                password = input("請輸入管理者密碼: ")
                if admin_list.login(username, password):
                    admin_panel(menu, order)
        elif admin_choice == '3':
            print("返回主選單")
            break
        else:
            print("選項無效，請重新選擇")
```

main.py

def admin_panel:

為管理者提供訂單管理、帳號管理和菜單管理功能

如果Order.all_orders為空:顯示無
訂單資料

如果Order.all_orders不為空:顯示
訂單編號、項目資訊和總金額

--- 管理者面板 ---

1. 檢視所有訂單
2. 顯示管理者帳號
3. AVLTree菜單管理
4. 登出

請選擇功能: 1

--- 所有訂單 ---

無訂單資料

--- 所有訂單 ---

訂單編號: 1

Tea: 1 x \$25

Burger: 2 x \$150

總金額: \$325

訂單編號: 2

Pizza: 1 x \$200

總金額: \$200

```
def admin_panel(menu, order):
    while True:
        print("\n--- 管理者面板 ---")
        print("1. 檢視所有訂單")
        print("2. 顯示管理者帳號")
        print("3. AVLTree菜單管理")
        print("4. 登出")

        choice = input("請選擇功能: ")
        if choice == '1':
            print("\n--- 所有訂單 ---")
            if not Order.all_orders:
                print("無訂單資料")
            for receipt in Order.all_orders:
                print(f"\n訂單編號: {receipt['id']}")
                for item in receipt['items']:
                    ice_level_str = item['ice_level'][1] if item['ice_level'] else ""
                    print(f"{item['item_name']}{ice_level_str}: {item['quantity']} x ${item['price']}")
                print(f"總金額: ${receipt['total']:.0f}")
            else:
                admin_list.display_admins()
            elif choice == '3':
                admin_avl_menu(menu)
            elif choice == '4':
                print("已登出")
                break
            else:
                print("選項無效，請重新選擇")
```

main.py

def customer_mode: 顧客模式的主介面

Enter: 完成點餐

Delete: 移除最後一項訂單

Back: 返回主選單

如果訂單為空，顯示“您尚未點餐”

```
----- 菜單 -----  
Burger: $150  
Coffee: $25  
Hotdog: $120  
Pizza: $200  
Salad: $80  
Tea: $25  
Water: $1  
請輸入欲點餐的項目 (按 Enter 完成  
點餐, 輸入 delete 可移除最後一項,  
輸入 back 回到主選單): |
```

```
----- 收據 -----  
Coffee*: 1 x $25  
Hotdog: 2 x $120  
總金額: $265  
訂單已儲存 (訂單編號: 3)  
感謝您的點餐
```

```
def customer_mode(menu, order):  
    print("\n--- 顧客模式 ---")  
    menu.display_menu()  
    while True:  
        item_name = input("請輸入欲點餐的項目 (按 Enter 完成點餐, 輸入 de  
  
        if item_name == "":  
            if order.head:  
                order.display_order()  
                order.save_receipt()  
                print("感謝您的點餐")  
            else:  
                print("您尚未點餐")  
            break  
  
        if item_name.lower() == 'back':  
            print("返回主選單")  
            break  
  
        if item_name.lower() == 'delete':  
            order.remove_last_item()  
            continue  
  
        node = menu.find_item(item_name)  
        if node:  
            try:  
                quantity = int(input(f"請輸入 {item_name} 的數量: "))  
                if quantity < 1:  
                    raise ValueError("數量不能為負數或零")  
                order.add_item(item_name, quantity)  
            except ValueError:  
                print("請輸入有效的數量。")  
        else:  
            print("菜單中沒有這個項目, 請重新輸入")
```

main.py

def main : 系統主程式

建立 Menu和Order 物件，預設加入一些菜單項目

輸入1：進入顧客模式

輸入2：進入管理者模式

輸入3：結束程式

```
--- 主選單 ---
1. 顧客模式
2. 管理者模式
3. 結束程式
請選擇模式： 1

--- 顧客模式 ---
```

```
--- 主選單 ---
1. 顧客模式
2. 管理者模式
3. 結束程式
請選擇模式： 2

--- 管理者模式 ---
```

```
--- 主選單 ---
1. 顧客模式
2. 管理者模式
3. 結束程式
請選擇模式： 3
程式已結束
```

```
def main():
    try:
        menu = Menu()
        menu.add_item("Salad", 80)
        menu.add_item("Hotdog", 120)
        menu.add_item("Burger", 150)
        menu.add_item("Pizza", 200)
        menu.add_item("Coffee", 25, is_drink=True)
        menu.add_item("Tea", 25, is_drink=True)

        order = Order(menu)

        while True:
            print("\n--- 主選單 ---")
            print("1. 顧客模式")
            print("2. 管理者模式")
            print("3. 結束程式")
            choice = input("請選擇模式： ")

            if choice == '1':
                customer_mode(menu, order)
            elif choice == '2':
                admin_mode(menu, order)
            elif choice == '3':
                print("程式已結束")
                sys.exit(0)
            else:
                print("輸入無效，請重新輸入")

        except Exception as e:
            print(f"程式執行時發生錯誤：{e}")

main()
```